

CLAHE — réglage et bonnes pratiques

Ce que le filtre fait vraiment, mesuré sur le corpus de validation

CLAHE — ce que le filtre fait vraiment

CLAHE découpe l'image en tuiles, égalise l'histogramme de chaque tuile séparément, et interpole entre les tuiles voisines pour qu'aucune frontière ne se voie. Le `clip_limit` borne la pente de chaque courbe : sans lui, l'égalisation d'une tuile plate étirerait son bruit sur toute la plage.

Ce qui compte devant un tribunal : chaque tuile applique une table de correspondance monotone, et l'interpolation bilinéaire entre tuiles est une moyenne pondérée de valeurs existantes. CLAHE ne crée donc pas de structure. C'est une propriété à écrire dans le rapport, parce que c'est la première question qui sera posée.

Ce qu'elle ne fait pas : récupérer de l'information écrêtée. Un pixel à 0 ou à 255 a perdu sa valeur d'origine. Aucun réglage ne la ramène.

Avant d'appliquer : lire l'histogramme

L'histogramme dit d'abord si CLAHE est le bon outil, ensuite avec quels réglages. C'est l'étape que l'on saute le plus souvent, et celle qui coûte le plus cher quand on la saute.

Ce que montre l'histogramme	Ce qu'il faut faire
Écrêtage à 0 ou 255 sur une part notable des pixels	Levels d'abord, et le noter au rapport : l'information est perdue, pas récupérable
Plage dynamique déjà pleine (p1→p99 proche de 255)	CLAHE n'apportera presque rien ; chercher ailleurs
Histogramme bimodal (scène de nuit avec sources lumineuses)	CLAHE globale gaspille son budget sur le fond ; recadrer sur la zone utile
Un bloc dont le sigma de bruit vaut exactement 0.00	Ce n'est pas « propre », c'est plat : la donnée y est détruite

Le dernier point mérite d'être souligné. Dans le rapport noise, un bloc à `sigma=0.00` à côté d'un bloc à `sigma=6.00` et une uniformité annoncée `uneven` décrivent une image dont les noirs sont bouchés dans une zone et dont le bruit est concentré dans une autre — typiquement un ciel nocturne. C'est exactement là que CLAHE amplifiera le plus, et pour rien.

Les fonctions de diagnostic existent déjà dans `src/filters/histogram.py` : `histogram_stats` (moyennes, percentiles, pourcentage d'écrêtage haut et bas) et `dynamic_range_used`.

Le coût réel du `clip_limit`

Facteur d'amplification du bruit mesuré sur trois trames CCTV réelles (grille 8×8, mode `lab`, bruit estimé par la méthode d'Immerkaer) :

image	σ initial	clip=1	clip=2	clip=3	clip=4	clip=6	clip=10
darkest	1.68	×1.49	× 1.91	×2.21	×2.46	×2.81	×3.20
flattest	1.59	×1.42	× 1.77	×2.02	×2.22	×2.50	×2.85
sharpest	4.63	×1.20	× 1.39	×1.50	×1.58	×1.70	×1.82

Deux enseignements :

1. **Au réglage par défaut de 2.0, le bruit est déjà presque doublé.** Ce n'est pas un réglage neutre.
2. **Le coût dépend de l'image, pas seulement du paramètre.** Une scène plate et sombre paye ×1.9 ce qu'une scène nette paye ×1.4. Un `clip_limit` fixe ne veut donc pas dire la même chose d'une pièce à l'autre — ce qui est un argument fort pour le dériver du bruit mesuré plutôt que de le laisser à un curseur.

Ordre de grandeur utilisable : **1.5 à 3 en vidéosurveillance**. Au-delà de 4, le bruit prend le dessus sur le gain de lisibilité.

La taille de tuile est un choix sur le sujet

Mesuré sur une cible synthétique à texture fine, contraste local relevé dans la zone d'intérêt :

grille	taille de tuile	contraste local	bruit global
2×2	192×128 px	14.50	7.95
4×4	96×64 px	16.73	8.07
8×8	48×32 px	17.80	8.27
16×16	24×16 px	18.56	8.39
32×32	12×8 px	20.50	10.47

Le contraste monte régulièrement ; le bruit décroche à 32×32, quand la tuile devient trop petite pour que son histogramme ait un sens.

La règle utilisable : une tuile doit faire environ un tiers de l'objet qui porte la question. Pour un visage de 60 px, il faut des tuiles d'environ 20 px. Sur une trame 1920, cela demanderait une grille de 96×96, ce qui est absurde — la bonne réponse est donc de **recadrer d'abord sur la zone d'intérêt**, puis d'appliquer CLAHE. Les tuiles tombent alors sur le sujet au lieu de dépenser leur budget sur le ciel.

Attention également : `clip_limit` est appliqué à l'histogramme d'une tuile, et le nombre de pixels par tuile dépend de la résolution. **Le même `clip_limit` sur la même scène en 720p et en 4K ne produit pas le même résultat.**

Le mode couleur

mode	usage
lab	Recommandé. CLAHE sur le canal L, teintes préservées
yuv	Équivalent en pratique ; travaille sur le Y de BT.601
luminance	Quasi identique à yuv (1/255 près) — voir l'annexe
hsv	Sur V ; peut saturer les couleurs vives
channelwise	À proscrire sur une pièce à conviction

channelwise égalise chaque canal RVB indépendamment, ce qui **déplace les couleurs**. On altère alors l'élément de preuve « couleur du véhicule » en croyant améliorer le contraste.

L'ordre de la chaîne : débruiter *après*

La règle habituelle est « débruiter d'abord ». **C'est faux avec cet outil**, et la mesure le dit. Cinq images assombries pour simuler une scène de nuit, deux niveaux de bruit, notées en PSNR contre la vérité terrain :

image	σ CLAHE seule	débruitage après	débruitage avant
building	4 30.82	32.18	31.98
building	10 23.44	25.62	24.70
lena	4 30.16	30.78	30.38
lena	10 22.45	23.90	22.63
baboon	4 30.17	29.97	30.40
baboon	10 22.49	23.63	22.77
sharpest	4 30.12	31.70	30.91
sharpest	10 22.65	25.02	23.15
flattest	4 29.12	30.39	29.52
flattest	10 21.49	23.66	21.67
moyenne	26.29	27.68	26.81

Gain moyen du débruitage : +1.39 dB après CLAHE, +0.52 dB avant. Gagnant dans 9 cas sur 10.

La raison est mécanique. `estimate_h` choisit sa force à partir du bruit qu'il *mesure*. Avant CLAHE il mesure $\sigma=5.5$ et applique $h=3.3$ — puis CLAHE double ce qui reste. Après CLAHE il mesure $\sigma=16.0$, applique $h=9.6$, et traite le bruit réellement présent.

Chaîne recommandée :

```
crop / roi    → cadrer sur ce qui porte la question
levels       → point noir, si l'histogramme le demande
clahe        → contraste local
nl_means_auto → débruitage, à la force mesurée après amplification
sharpen      → léger, en dernier
```

Cet ordre respecte aussi la discipline générale : correction géométrique avant rehaussement (corriger la géométrie après avoir amplifié le contraste revient à interpoler des pixels déjà amplifiés), et présentation en dernier.

CLAHE dirigée par l'histogramme

CLAHE est déjà locale dans son **analyse** — chaque tuile a sa propre table — mais globale dans sa **contrainte** : un seul `clip_limit` pour toutes les tuiles. Le ciel plat et bruité et le visage texturé reçoivent la même autorisation d'amplifier. C'est une incohérence réelle, et la corriger est la piste d'amélioration la plus prometteuse.

Le principe

Une carte de demande, une valeur par tuile, croisant deux facteurs normalisés par rang à l'intérieur de l'image :

- **besoin** — l'histogramme de la tuile est-il comprimé ? Une tuile déjà contrastée n'a rien à gagner.
- **sûreté** — écart-type sur sigma de bruit. Ce qu'on amplifierait est-il du signal ? Une tuile terne *parce qu'elle est vide* a un mauvais score.

C'est le croisement des deux qui désigne le cas intéressant en forensique : **un détail faible mais réel**, un visage dans l'ombre.

Le piège à éviter

Calculer plusieurs CLAHE à des forces différentes et les mélanger avec une carte de poids **ne réalise pas** une CLAHE à clip variable. Moyenner deux courbes tonales différentes donne une courbe plus plate que chacune des deux. Mesuré : le contraste global tombait à **×0.929**, en dessous de l'image d'entrée, donc moins bon que n'importe quelle CLAHE uniforme.

Il faut faire varier le clip **à l'intérieur** de l'algorithme, sur les tables de correspondance, avant l'interpolation.

La validation avant la mesure

Une réimplémentation de CLAHE ne vaut rien tant qu'elle n'a pas été comparée à une référence **à clip uniforme**, cas où le résultat doit être identique :

LUT seule (grille 1×1)	: écart moyen 0.000	max 0	← exacte
grille 8×8 – intérieur	: écart moyen 0.274	max 2	← arrondi
grille 8×8 – bord	: écart moyen 0.210	max 1	

Ce contrôle a révélé deux bugs : une normalisation de table qui ne suivait pas celle d'OpenCV, et un calcul d'indice qui bornait avant de prendre la tuile voisine — donc mélangeait deux tuiles sur la première demi-tuile. Avant correction, l'écart au bord était de **8.41 en moyenne, 66 au maximum**. Sans ce contrôle, ces écarts auraient été attribués au « bénéfice » de la méthode.

Le résultat, à contraste égal

CLAHE uniforme réglée pour produire exactement le même contraste global :

image	bruit dirigée	bruit uniforme	gain
brightest	×1.772	×3.618	+51.0 %

building	×1.460	×2.524	+42.1 %
darkest	×1.721	×2.003	+14.1 %
sharpest	×1.409	×1.571	+10.3 %
most_blown	×1.660	×1.731	+4.1 %
flattest	×1.605	×1.596	−0.6 %
lena	×1.639	×1.625	−0.8 %
softest	×1.861	×1.574	−18.2 %

Moyenne **+12.7 %**, 5 images sur 8.

Ce qui reste ouvert

Ce qui sépare les gains des pertes n'est pas identifié. L'hypothèse d'une redondance des deux facteurs sur les images floues ne tient pas : `softest` et `brightest` ont une corrélation besoin/sûreté quasi identique (+0.80 et +0.78) et finissent à −18 % et +51 %.

Tant que ce n'est pas expliqué, **c'est un résultat prometteur, pas un résultat acquis**. Livrer cela comme comportement par défaut ferait perdre 18 % sur certaines pièces sans savoir lesquelles.

Bonnes pratiques, dans l'esprit d'Amped FIVE

Ces principes portent sur la conception d'un logiciel forensique. Ils sont à recouper avec la documentation d'Amped avant d'être cités dans un rapport.

Le filtrage sélectif

On n'applique pas un rehaussement à toute l'image : on l'applique à la zone qui porte la question — la plaque, le visage, la main — et on laisse le reste intact, **parce que le reste n'est pas ce qu'on démontre**.

C'est aussi la vraie réponse au problème de l'histogramme bimodal : appliquée à la zone utile seule, CLAHE travaille sur l'histogramme qui intéresse.

Deux exigences : la région doit être consignée dans le rapport, et **la transition doit être adoucie**. Une couture nette sur une pièce à conviction est une question à l'audience.

Un rapport qui explique, pas seulement qui liste

Un rapport forensique est lu par quelqu'un qui n'est pas analyste d'image. Chaque filtre doit y être accompagné d'une description de ce qu'il fait, en langue courante — pas seulement de son nom et de ses paramètres.

L'aperçu par balayage de paramètres

Ne jamais deviner un paramètre : produire la planche des alternatives, la regarder, et choisir celle qu'on peut justifier. Compte tenu du fait que le coût en bruit d'un `clip_limit` donné varie d'un facteur 1.4 à 1.9 selon l'image, un opérateur devant un curseur reprendra la valeur par défaut ; un opérateur devant la planche choisira.

La vue différence

C'est ainsi qu'on **voit** l'amplification du bruit au lieu de la mesurer. Sur une scène de nuit, une carte de différence fait apparaître immédiatement que l'essentiel de l'action du filtre se passe dans le ciel — c'est-à-dire nulle part où on la voulait.

L'ordre de la chaîne comme règle

Chargement et intégrité, puis correction géométrique, puis rehaussement, puis présentation. L'ordre n'est pas un goût.

Ne pas générer d'information

Pas de rehaussement génératif qui invente un visage plausible. L'agrandissement doit rester de l'interpolation ; la super-résolution doit combiner des trames réellement acquises. C'est une propriété à protéger explicitement.

Aide-mémoire

1. Lire l'histogramme **avant** de toucher au filtre. Écrêtage → `levels`, et le noter.
 2. Recadrer sur ce qui porte la question, pour que les tuiles tombent sur le sujet.
 3. `clip_limit` entre 1.5 et 3. Vérifier le coût en bruit sur *cette* image, pas en général.
 4. Grille telle qu'une tuile fasse environ un tiers de l'objet.
 5. Mode `lab`. Jamais `channelwise` sur une pièce à conviction.
 6. Débruiter **après** CLAHE, pas avant.
 7. Consigner tous les paramètres — le preset JSON rejoue la chaîne à l'identique.
 8. Comparer avant/après en vue différence, pas seulement à l'œil.
-

Annexe : défauts relevés dans le code

Emplacement	Constat
<code>src/filters/clahe.py:100</code>	Ligne morte : <code>result</code> est écrasé deux lignes plus bas — corrigé (sortie inchangée au bit près, vérifiée sur 10 images)
<code>src/filters/clahe.py:96</code>	Le mode <code>luminance</code> est <i>presque</i> un doublon de <code>yuv</code> : <code>RGB2GRAY</code> et le <code>Y</code> de <code>RGB2YUV</code> sont la même combinaison BT.601, mais les arrondis différent . Repris sur l'ensemble du corpus (29 images, 35,6 M pixels) : écart de 1 sur 400 pixels (0,001 %), que CLAHE amplifie ensuite jusqu'à 4/255 en sortie. La mesure initiale — « écart maximal 0 » — portait sur une seule trame et ne se généralise pas. Fusionner les deux branches changerait donc ce que rejoue un preset existant : corrigé en documentant l'écart, pas en fusionnant
<code>src/filters/clahe.py:60,66</code>	Le 8 bits est forcé en entrée, alors qu'OpenCV accepte le 16 bits. Sur une source 10 ou 12 bits, la précision est jetée juste avant l'étape qui étire le contraste
<code>src/filters/roi.py:94</code>	<code>apply_to_roi</code> applique un filtre à une région et n'est pas enregistré dans <code>FILTER_REGISTRY</code> : aucune interface ne peut l'atteindre. Bords nets, sans adoucissement
<code>src/filters/clahe.py:117</code>	<code>apply_clahe_grid</code> produit la planche d'aperçu et n'est pas enregistré non plus
<code>src/core/report.py:72</code>	Le rapport écrit nom, module, horodatage et paramètres, mais aucune description — alors que chaque entrée de <code>FILTER_REGISTRY</code> en porte une
<code>src/gui/widgets.py:92</code>	<code>VIEW_MODES</code> n'offre pas de vue différence

Méthode

Mesures faites sur le corpus de validation : trames CCTV réelles (`validation/corpus/cctv/`) et images de référence (`validation/corpus/reference/`), redimensionnées en 512×384, grille 8×8, mode `lab`. Le contraste est l'écart-type du canal L de LAB ; le bruit est l'estimateur d'Immerkaer (`estimate_noise`). Les comparaisons « à contraste égal » interpolent la courbe de la CLAHE uniforme pour trouver le `clip_limit` donnant le même contraste que la version dirigée, puis comparent le bruit.

Le tableau de la taille de tuile utilise une cible synthétique à texture contrôlée ; tous les autres tableaux portent sur des images réelles.

Les prototypes et le script de comparaison sont dans `validation/prototypes/` : `clahe_lut.py` (CLAHE à clip variable), `guided_clahe.py` (carte de demande), `compare_guided.py`.